

Exercise 1: Implementing the Singleton Pattern

Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class:

- Create a class named `Logger` that has a private static instance of itself.
- Ensure the constructor of `Logger` is private.
- Provide a public static method to get the instance of the `Logger` class.

3. Implement the Singleton Pattern:

- Write code to ensure that the `Logger` class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- Create a test class to verify that only one instance of `Logger` is created and used across the application.

Solution:

```
public class SingletonDemo {
    public static void main(String[] args) {
        SystemConfig primaryConfig = SystemConfig.fetchInstance();
        SystemConfig secondaryConfig = SystemConfig.fetchInstance();

        primaryConfig.displayMessage("Application Initialized");
        System.out.println(primaryConfig == secondaryConfig);
    }
}

class SystemConfig {
    private static SystemConfig instance;

    private SystemConfig() {
        System.out.println("Configuration Instance Created");
    }

    public static SystemConfig fetchInstance() {
        if (instance == null) {
            instance = new SystemConfig();
        }
        return instance;
    }

    public void displayMessage(String message) {
        System.out.println(message);
    }
}
```

Output:

```
C:\Users\23BCE8750\Desktop\Rehaan\Cognizent>javac SingletonWrapper.java
C:\Users\23BCE8750\Desktop\Rehaan\Cognizent>java SingletonWrapper
Log created
App started
true
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. **Create a New Java Project:**
 - Create a new Java project named **FactoryMethodPatternExample**.
2. **Define Document Classes:**
 - Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.
3. **Create Concrete Document Classes:**
 - Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.
4. **Implement the Factory Method:**
 - Create an abstract class **DocumentFactory** with a method **createDocument()**.
 - Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.
5. **Test the Factory Method Implementation:**
 - Create a test class to demonstrate the creation of different document types using the factory method.

Solution:

```
public class FactoryPatternDemo {
    public static void main(String[] args) {
        FileCreator creatorA = new DocCreator();
        FileCreator creatorB = new PortableCreator();
        FileCreator creatorC = new SheetCreator();
        FileItem fileA = creatorA.createDocument();
        FileItem fileB = creatorB.createDocument();
        FileItem fileC = creatorC.createDocument();
        fileA.open();
        fileB.open();
        fileC.open();
    }
}

interface FileItem {
    void open();
}

class DocFile implements FileItem {
    @Override
    public void open() {
        System.out.println("Open Word");
    }
}

class PortableFile implements FileItem {
    @Override
    public void open() {
        System.out.println("Open PDF");
    }
}

class SheetFile implements FileItem {
    @Override
    public void open() {
        System.out.println("Open Excel");
    }
}
```

```

abstract class FileCreator {
    public abstract FileItem createDocument();
}
class DocCreator extends FileCreator {
    @Override
    public FileItem createDocument() {
        return new DocFile();
    }
}
class PortableCreator extends FileCreator {
    @Override
    public FileItem createDocument() {
        return new PortableFile();
    }
}
class SheetCreator extends FileCreator {
    @Override
    public FileItem createDocument() {
        return new SheetFile();
    }
}

```

Output:

```

C:\Users\23BCE8750\Desktop\Rehaan\Cognizent>javac FactoryPatternDemo.java
C:\Users\23BCE8750\Desktop\Rehaan\Cognizent>java FactoryPatternDemo
Open Word
Open PDF
Open Excel

```